

Style Transfer Report

Name – Vishruth Byaramadu Lokesh

Course: CSCI 611 – Spring 2025

Assignment: Style Transfer – Image Transformation with VGG19

1. Introduction

In this project, I implemented a style transfer algorithm that blends the content of one image (e.g., a photograph of a tiger) with the style of another (e.g., an artistic painting). I followed the guidelines provided in the assignment and the structure outlined in the Jupyter Notebook. My goal was to experiment with various hyperparameters and analyze how each setting affected the final output.

The project involved:

- Completing the style transfer exercise by filling in the necessary code sections.
- Experimenting with different hyperparameters (such as style weight, content weight, number of iterations, layer style configurations, and optimizers) to observe their effects on the generated image.
- Documenting my implementation process, experimental findings, and including visualizations to support my analysis.

2. Implementation Overview

2.1. Preprocessing and Model Setup

- **Image Loading and Preprocessing:**
I created a helper function `load_image` to read the content and style images (e.g., `tiger.jpg` and `style.jpg`). The images are resized (with a maximum dimension of 400 pixels) and normalized using the mean and standard deviation values consistent with the VGG19 training process.
- **Model Initialization:**
I loaded the pretrained VGG19 model (only the feature extraction part) and froze its parameters, ensuring that only the target image (initialized as a clone of the content image) is updated during the optimization process.
- **Device Setup:**
I configured the device to use a GPU if available, which speeds up the processing.

2.2. Feature Extraction

- **Content and Style Features:**
I implemented the function `get_features` to pass an image through the VGG19 network and extract feature maps from specific layers. For the content representation, I primarily used

the 'conv4_2' layer, and for the style representation, I used multiple layers (e.g., 'conv1_1', 'conv2_1', 'conv3_1', 'conv4_1', and 'conv5_1').

- **Gram Matrix Computation:**

The function `gram_matrix` calculates the Gram matrix for a given feature map. This matrix captures the correlations between feature channels and is used to represent the style of the image.

2.3. Loss Functions

I computed two types of losses during the optimization:

- **Content Loss:**

The content loss is computed as the mean squared error between the feature representation of the target image and that of the content image (specifically, from the 'conv4_2' layer).

- **Style Loss:**

For each style layer, I calculated the style loss as the mean squared error between the Gram matrix of the target image's features and that of the style image. Each layer's contribution was weighted by coefficients defined in dictionaries like `style_weights` and normalized by the dimensions of the feature maps.

- **Total Loss:**

The overall loss is a weighted sum of the content and style losses, where `content_weight` (α) and `style_weight` (β) control their relative contributions.

2.4. Optimization Strategies

I explored two optimizers for updating the target image:

- **Adam Optimizer:**

I used the Adam optimizer in a loop where the target image was updated over a specified number of iterations. The learning rate and iteration count were adjustable, and I experimented with different settings to observe their impact on the final result.

- **L-BFGS Optimizer:**

I also employed the L-BFGS optimizer, which uses a closure function to perform updates. Although it converged quickly in fewer iterations, I needed to fine-tune the learning rate (set to 0.1) and the number of iterations to get the desired effect.

3. Hyperparameter Experiments and Results

I experimented with several hyperparameters. Below, I detail the settings I used and the outcomes I observed. In your report, be sure to insert your result images where indicated.

Content Image



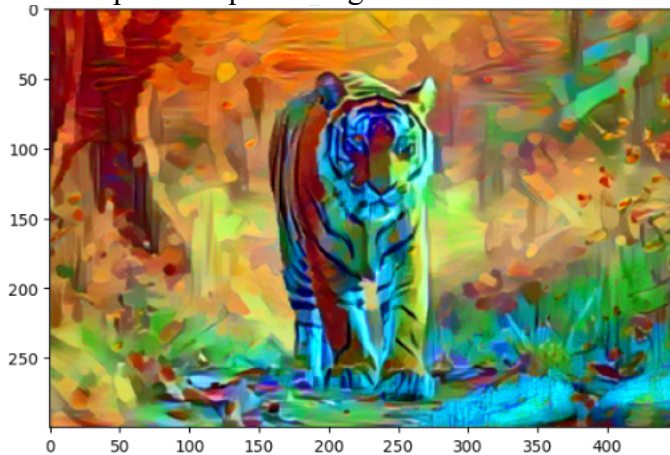
Style Image



3.1. Style Weight (style_weight)

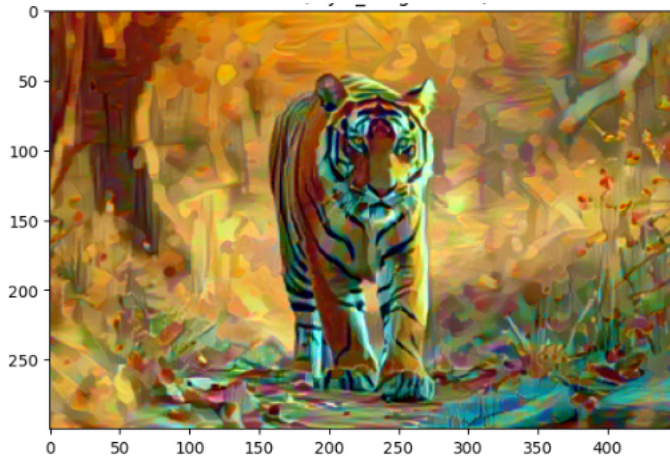
- **High Style Weight (e.g., 1e6 or 1e8):**
By increasing the style weight, I forced the target image to adopt a stronger influence from the style image. For instance, with `style_weight=1e8` in the bold configuration, the stylized output exhibited very pronounced style patterns. However, this sometimes came

at the expense of preserving fine content details.



- **Low Style Weight (e.g., 0.1 or 1):**

A lower style weight resulted in the target image retaining more of the original content. With `style_weight=0.1`, the output stayed closer to the content image, with subtler stylistic influences.

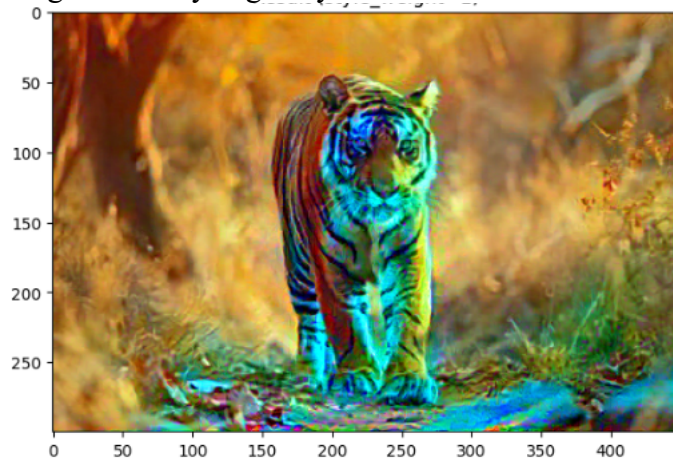


3.2. Content Weight (`content_weight`)

- **High Content Weight (e.g., 1000):**

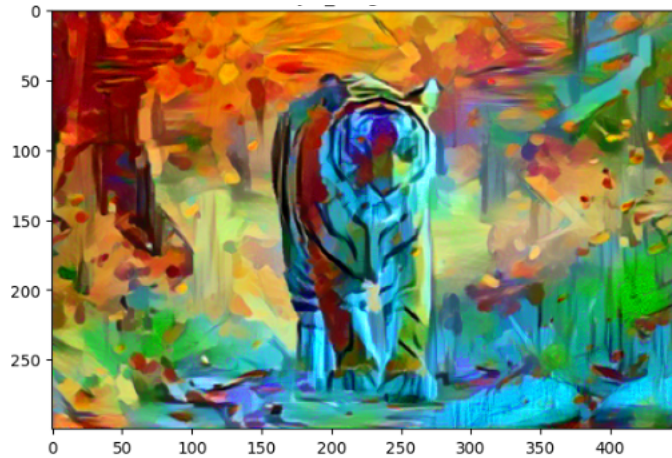
When I increased the content weight, the algorithm prioritized preserving the original image's structure. With `content_weight=1000`, the final image closely resembled the content

image with only slight stylistic modifications.



- **Low Content Weight (e.g., 1):**

Lowering the content weight allowed the style to dominate the output, resulting in a more dramatic artistic transformation that deviated further from the original content.

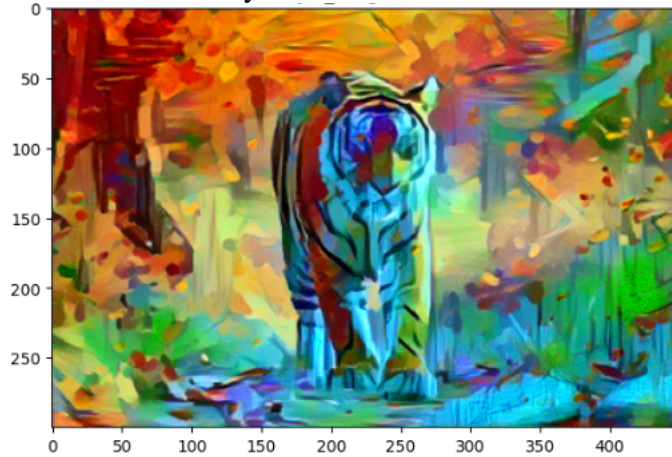


3.3. Number of Steps (Iterations)

- **Higher Iterations (e.g., 5000 steps):**

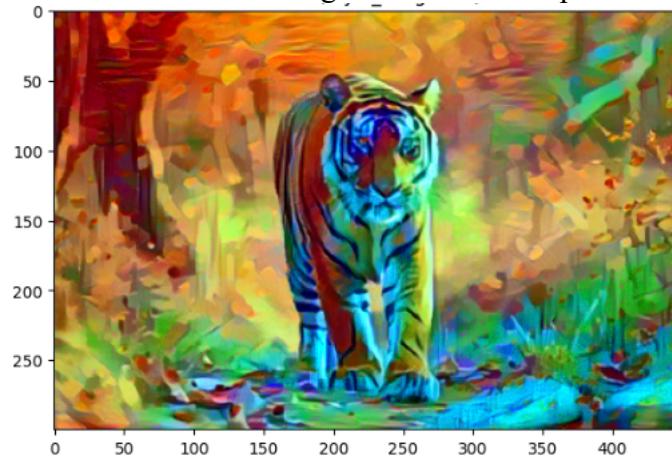
I found that more iterations generally allowed for better convergence of the optimization process, yielding a more refined image. However, too many iterations sometimes

introduced over-stylization or artifacts.



- **Lower Iterations (e.g., 1000 steps):**

Fewer iterations resulted in an output where the style was only partially applied, and much of the content image's structure was preserved.

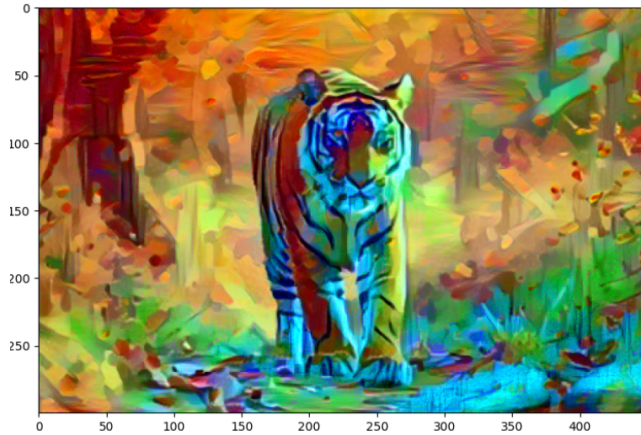


3.4. Layer Style Configurations

I experimented with different preset configurations for style weights:

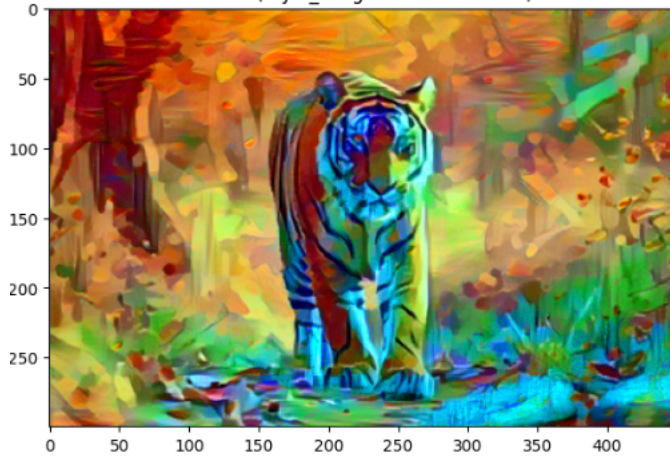
- **Default:**

The default configuration uses higher weights for earlier layers (e.g., conv1_1: 1.0) and gradually lower weights for later layers (e.g., conv5_1: 0.1), achieving a balanced mix of content and style.



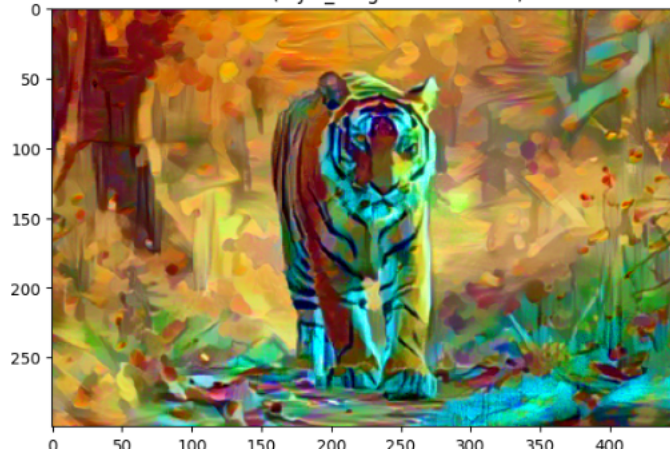
- **Bold:**

In the bold configuration, I set the weights such that early layers (e.g., conv1_1 and conv2_1) had high values, resulting in larger, more noticeable style artifacts. This setting produced a more dramatic style transfer effect.



- **Fine:**

The fine configuration gradually increases style weights from the first to the last layer (e.g., conv1_1: 0.2 to conv5_1: 1.0). This setting emphasizes more detailed, subtle style features while still preserving the overall content structure.



3.5. Optimizer Choice

- **Adam Optimizer:**
I primarily used the Adam optimizer with varying learning rates (e.g., 0.003 and 0.01) and iteration counts. Its adaptive learning rate provided smooth updates over thousands of iterations, making it suitable for gradual evolution of the target image.
- **L-BFGS Optimizer:**
The L-BFGS optimizer converged more quickly with fewer iterations (e.g., 50 steps). While it produced distinct visual characteristics in the final image, it required careful tuning of the learning rate and iteration count.



4. Discussion

My experiments led to several key insights:

- **Balancing Content and Style:**
The right balance between content and style weights is critical. A very high style weight can overwhelm the content, while a high content weight preserves too much of the original image. Finding the optimal balance is essential for achieving the desired artistic effect.
- **Iteration Count:**
While increasing the number of iterations generally improves the result, the benefits tend to plateau after a certain point. Experimenting with different iteration counts helped me identify a point where further iterations yielded minimal improvements.
- **Layer Selection Impact:**
Different layer style configurations (default, bold, fine) significantly influenced the final output. The bold configuration, for example, produced dramatic style features, whereas the fine configuration resulted in more subtle style enhancements.
- **Optimizer Effects:**
The choice of optimizer (Adam vs. L-BFGS) influenced both the speed of convergence and the characteristics of the output image. Adam was more flexible for long training sessions, while L-BFGS provided rapid convergence with fewer steps.

5. Conclusion

In this project, I implemented a style transfer model using a pretrained VGG19 network and explored several hyperparameters, including style weight, content weight, number of iterations, layer style configurations, and optimizer choice. Each of these parameters had a direct impact on the balance between preserving the original content and imposing the new style:

- **Style weight** controlled the intensity of the style influence.
- **Content weight** determined how much of the original content was retained.
- **Iteration count** affected the level of refinement and convergence of the target image.
- **Layer style configurations** provided different balances of fine versus bold stylistic details.
- **Optimizer choice** influenced the speed and nature of the optimization process.

By systematically experimenting with these parameters, I was able to generate a variety of artistic outputs, each demonstrating a unique interplay between content and style. This report documents my implementation and analysis, offering insights into the mechanics of style transfer and serving as a foundation for future work in artistic image synthesis.